WEB DEVELOPMENT – III

Node.js Modules

Session 4 — Study Notes

Topic: Modules, Core/Local/Third-Party, CommonJS vs ES Modules

Instructor: Love Porwal
K.R. Mangalam University — Sohna Road, Gurugram

1. What Are Modules?

A LEGO city is never built from one giant block. It's built from thousands of small, standard pieces — each one simple and self-contained — combined in different ways to build a house, a car, or a tree. Software works the same way, and Node.js enforces this by design: every file is automatically its own module, a self-contained unit of code with a private scope.

This privacy is not optional or something a developer has to remember to set up — it's the default. A variable declared in one file simply does not exist in another file, unless it's deliberately exported. Two files can both declare a variable named total without the slightest conflict, because each file's variables live in their own sealed scope.

```
// fileA.js
const total = 100;
// 'total' only exists inside fileA.js

// fileB.js
const total = 500;
// a completely different 'total' — no clash, no shared state
```

Why This Matters in Practice

Breaking code into modules gives five concrete benefits, each solving real problem large codebases run into:

- **Encapsulation —** a module hides its implementation details and exposes only a deliberate, chosen API to the outside world.
- **Reusability —** a module is written once and imported into as many files as needed, instead of being copy-pasted repeatedly.
- **Organization —** code is broken into logical units, so a project's structure mirrors its actual responsibilities.
- **Scope isolation —** variables never leak into a shared global namespace, which is exactly what causes hard-to-trace bugs in large, unmodularized codebases.
- **Clear dependencies —** every require() or import statement at the top of a file states exactly what that file needs, making the codebase easier to read and reason about.

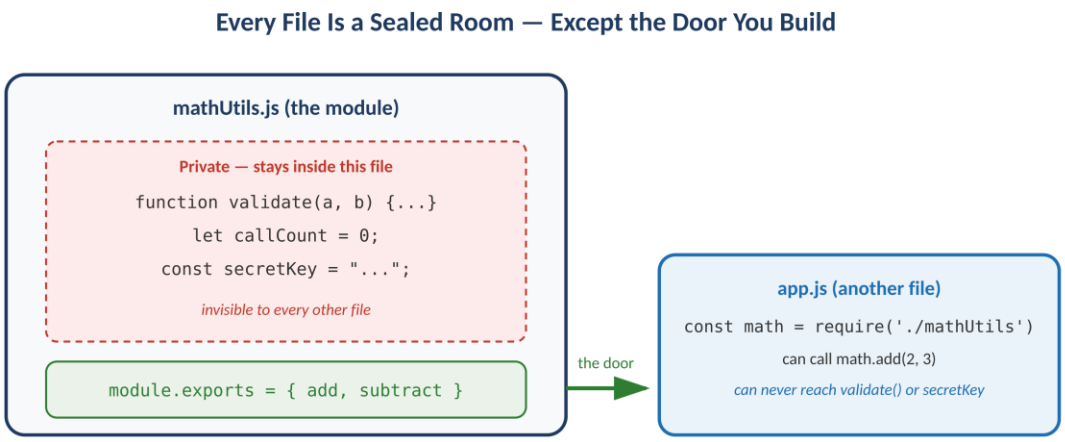

Figure 1.1 — A module's private code stays sealed inside it; only what's explicitly exported can be reached from outside.

Without modules, all of this collapses. Every variable would live in one shared global space, meaning two unrelated files could silently overwrite each other's data just by choosing the same name. Code could not be meaningfully reused, since there would be no clean boundary to import across. Maintaining or testing such a codebase becomes progressively harder as it grows — which is precisely why modules matter for scalability, maintainability, reusability, and testability alike.

2. Types of Modules

Every module used in a Node.js project falls into exactly one of three categories, distinguished by where the code actually comes from:

- **Core Modules —** built directly into Node.js itself; nothing to install.
- **Local Modules —** files written inside your own project, imported with a relative path.
- **Third-Party Modules —** installed from the npm registry with npm install.

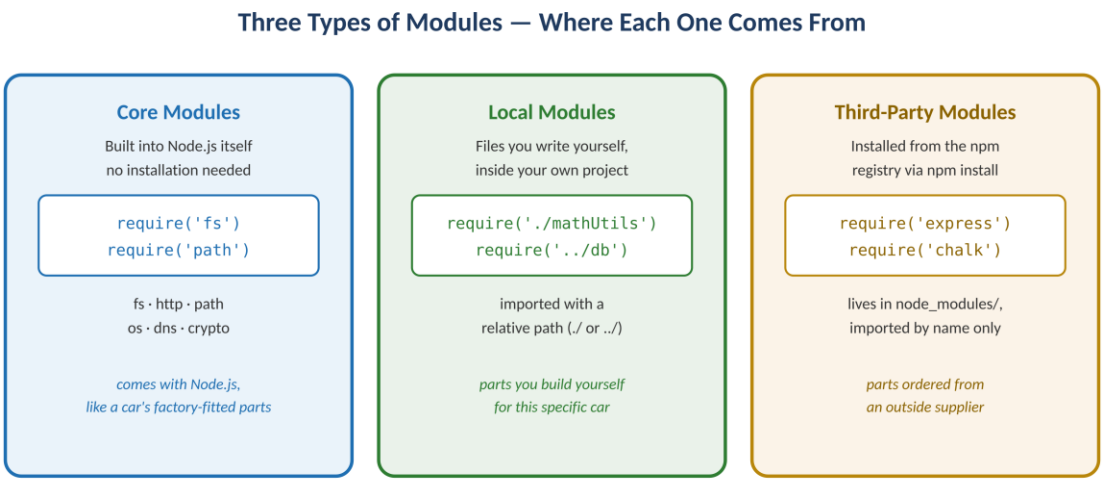

Figure 2.1 — Same require() syntax, three very different sources.

3. Core Modules

Core modules are compiled directly into the Node.js binary itself — there is no npm install step, and no internet connection is required to use them. They are imported exactly like any other module, with require(), but by name only, never by path:

```
const fs = require('fs');
const http = require('http');
const path = require('path');
```

Module	Use Case
fs	File system read, write, and delete operations
http	Create HTTP servers and clients
path	File path manipulation utilities
os	Operating system information
dns	Domain name resolution

A short, realistic example — reading a file's contents using the fs core module:

```
const fs = require('fs');

fs.readFile('notes.txt', 'utf8', (err, data) => {
  if (err) {
    console.error('Could not read file:', err.message);
    return;
  }
```

```
  console.log(data);
});
```

4. Local Modules

Local modules are files written inside a project, imported using a relative path — starting with ./ for the same folder, or ../ to step up into a parent folder. That leading dot is what tells Node.js "look in my own project," as opposed to checking installed packages or core modules.

Creating one starts with writing the code and deciding what to export:

```
// mathUtils.js

function validate(a, b) {
  if (typeof a !== 'number' || typeof b !== 'number') {
    throw new Error('Both arguments must be numbers');
  }
}

// Public functions — these will be exported
function add(a, b)      { validate(a, b); return a + b; }
function subtract(a, b) { validate(a, b); return a - b; }
function multiply(a, b) { validate(a, b); return a * b; }
function divide(a, b) {
  if (b === 0) throw new Error('Cannot divide by zero');
  return a / b;
}

module.exports = { add, subtract, multiply, divide };
```

Notice that validate is never included in module.exports. It still runs, and other functions in the same file can still call it, but it is completely invisible to any file that imports mathUtils — exactly the encapsulation described in Section 1. Using this module elsewhere looks like:

```
// app.js
const math = require('./mathUtils');   // relative path

console.log(math.add(10, 5));          // 15
console.log(math.subtract(10, 5));     // 5
console.log(math.multiply(10, 5));     // 50
console.log(math.divide(10, 5));       // 2

// Destructuring import — pulling out only what's needed
const { add, multiply } = require('./mathUtils');
console.log(add(3, 4));               // 7

// Error handling
try {
```

```
  console.log(math.divide(10, 0));    // throws error
} catch (err) {
  console.error('Error:', err.message);
}
```

Module Caching — require() Only Runs a File Once

The first time any file calls require('./mathUtils'), Node.js runs mathUtils.js from top to bottom and caches the resulting module.exports object. Every subsequent require('./mathUtils') anywhere else in the project — even from a completely different file — returns that same cached object instantly, without re-running the file's code.

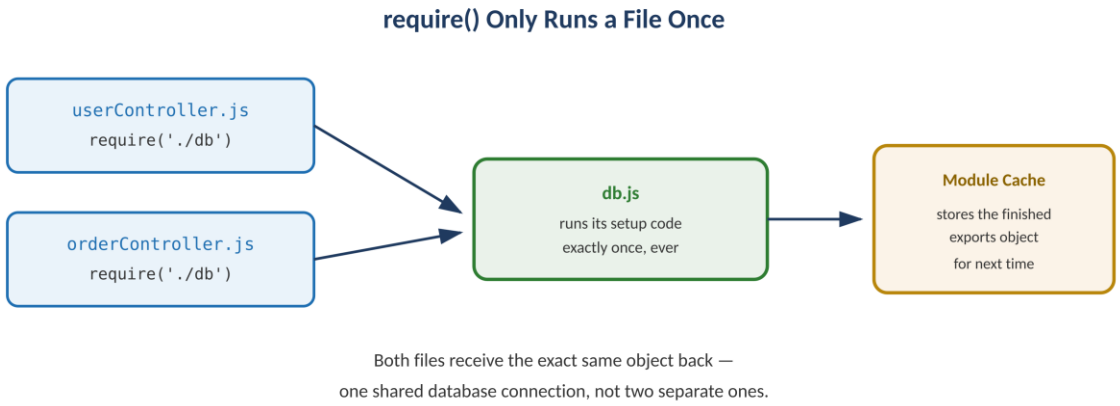

Figure 4.1 — Two different files requiring the same module both receive the identical, already-built object.

This is far more than a performance optimization. It's the mechanism that makes a shared database connection or a shared configuration object work correctly: if userController.js and orderController.js both require a db.js module that opens a database connection, that connection is opened exactly once and shared by both — not duplicated. This pattern, where a module effectively behaves like a single shared instance across an entire application, is known as the singleton pattern, and it falls out naturally from how require() caching works.

5. Third-Party Modules

Third-party modules are packages installed from the npm registry using npm install, and once installed, they are physically stored inside the project's node_modules folder. Importing one uses exactly the same require() syntax as a core module — Node.js automatically checks node_modules when it doesn't recognize a name as a core module and the string doesn't start with ./ or ../.

As a working example, the chalk package adds color to terminal output:

```
// terminal:
npm install chalk@4   // v4 uses CommonJS — matches require() syntax
```

```
// app.js
const chalk = require('chalk');

console.log(chalk.green('Success!'));
console.log(chalk.red.bold('Error!'));
```

Notice the @4 pinned in the install command — this is deliberate, since newer major versions of chalk switched to ES Modules only (covered in Section 7), and would not work with a plain require() call.

6. Module Export/Import Methods — The CommonJS Pattern

CommonJS is the module system Node.js has used since its very beginning, and it remains the default today. Its two halves are require(), used to import a module, and module.exports, used to export one. Loading is synchronous — when a require() line executes, Node.js pauses, fully loads that file, and only then continues to the next line — which is simple to reason about, though it does mean a require() call blocks briefly while the file loads.

CommonJS dominates the Node.js ecosystem for a very concrete reason: it has been the standard since the beginning, so essentially every package ever published to npm is written using it. Express, MongoDB drivers, and virtually every major library expect to be required with CommonJS syntax, and a project's own code typically needs to match that same syntax for everything to interoperate smoothly.

```
// math.js — creating a module
function add(a, b) { return a + b; }
function multiply(a, b) { return a * b; }

module.exports = { add, multiply };

// app.js — importing the module
const math = require('./math');
console.log(math.add(3, 4));        // 7
console.log(math.multiply(3, 4));   // 12

// Core modules use the exact same syntax
const fs = require('fs');
```

7. CommonJS vs ES Modules

ES Modules (ESM) is JavaScript's own official module system — the same import and export syntax used natively in browsers, now also supported inside Node.js. A file opts into it either by using the .mjs extension, or by adding "type": "module" to package.json:

```
// math.mjs (or set "type": "module" in package.json)
export function add(a, b) { return a + b; }
export function multiply(a, b) { return a * b; }
export default { add, multiply };

// app.mjs — importing an ES module
import { add, multiply } from './math.mjs';
import math from './math.mjs';   // default export

console.log(add(3, 4));             // 7
console.log(multiply(3, 4));        // 12
```

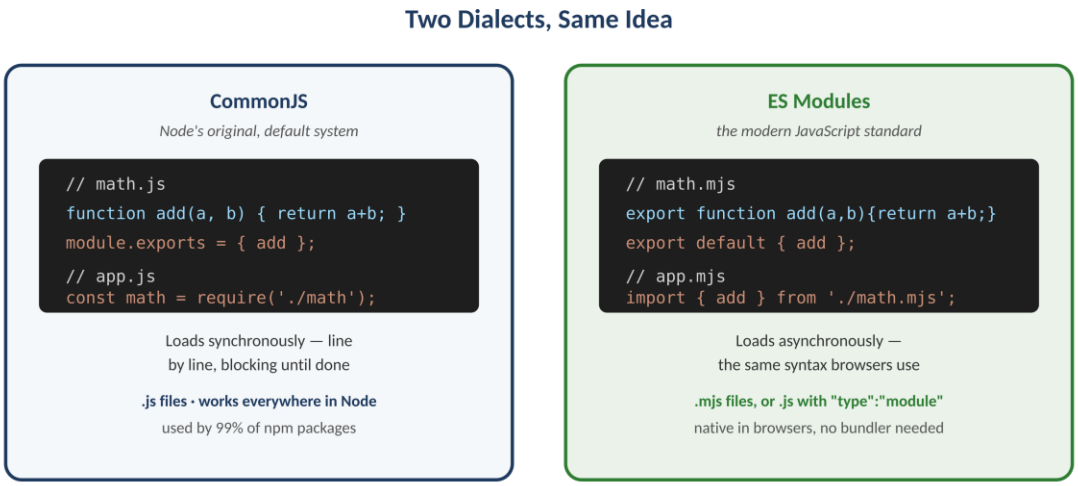

Figure 7.1 — Same two functions, exported and imported in each of the two dialects.

Feature	CommonJS	ES Modules
Syntax	require() / module.exports	import / export
File extension	.js or .cjs	.mjs or .js (with type: module)
Loading	Synchronous	Asynchronous
Default in Node	Yes (traditional)	Supported since Node 12+
Browser support	No (needs a bundler)	Yes (native)

ESM has not replaced CommonJS as Node's default, and the reason is almost entirely historical rather than technical: CommonJS came first, and more than two million npm packages have already been written against it. Switching the default would break an enormous fraction of the existing JavaScript ecosystem overnight. Node.js fully supports ESM today for anyone who wants it — new projects are free to use import and export throughout — but CommonJS remains the default specifically to preserve backward compatibility with two decades of existing packages.

8. Key Takeaways

require('module-name') without any leading dot imports a core module (or an installed package from node_modules), while require('./filename') with a leading dot imports a local file from within the project. The presence or absence of that ./ is what tells Node.js where to look.

Module caching means that once a file has been required anywhere in a project, its result is cached and reused for every future require() of that same file — the file's code does not run again. This is what allows a shared resource, such as a single database connection, to be created once and reused consistently everywhere it's imported.

ESM is not yet Node's default because CommonJS came first, and npm carries roughly two decades of packages written against it. Node.js supports ESM today via .mjs files or "type": "module", and new projects are free to adopt it, but changing the default outright would break an enormous existing ecosystem.

9. Summary

- Every file in Node.js is automatically its own module, with a private scope — nothing leaks globally unless deliberately exported.
- Modules exist in three flavors: core (built into Node.js), local (written inside the project), and third-party (installed via npm).
- A local module exports its public API with module.exports, and anything left out stays private to that file.
- require() caches results — a module's file runs only once, no matter how many other files import it.
- CommonJS (require/module.exports) is Node's default and is what almost every npm package still uses; ES Modules (import/export) is the modern JavaScript-native alternative, supported but not yet default.

10. References

- Node.js Official Documentation — modules, require(), the module system, and CommonJS/ESM interoperability.
- W3Schools — Node.js Tutorial: beginner-friendly explanations with hands-on examples.
- freeCodeCamp — Node.js Course: free, project-based learning for Node.js, Express.js, and REST APIs.
- npm Docs — official npm documentation and CLI reference (docs.npmjs.com).